

|             |                         |         |                            |
|-------------|-------------------------|---------|----------------------------|
| NIM         | : 2411522016            | Tanggal | : Jumat, 04 April 2026     |
| Nama        | : Dea Arlia             | Asisten | : 1. Allyah Shiera Gunawan |
| Mata Kuliah | : Praktikum Data Mining |         | 2. Fahri Zamzami           |
| Modul       | : 03                    |         | 3. Mashia Zhavira Septyana |
| Kelas       | : B                     |         | 4. Muhammad Rafi Asytar    |
|             |                         |         | 5. Fadhilla Firma          |
|             |                         |         | 6. MHD Hasbi               |
|             |                         |         | 7. Zhahra Idhya Astwoti    |
|             |                         |         | 8. Olivia Delisa           |

### Instruksi “Klasifikasi (Decision Tree & SVM)”

1. Dataset "ikan.csv" terdiri dari 4 atribut yaitu panjang tubuh, lebar tubuh, panjang sirip, dan lebar sirip. Terdapat beberapa spesies ikan sebagai kelas target pada dataset ini. Data ini dipakai untuk masalah klasifikasi, di mana kita bisa memprediksi spesies dari sebuah ikan berdasarkan atribut-atribut yang diberikan. Tahapan yang ada pada tugas ini antara lain:

1. Load dataset 'ikan.csv' dan ubah ke dalam dataframe.

```
[2]: # Membaca file CSV
df = pd.read_csv('ikan.csv')

# Menampilkan 5 baris pertama untuk melihat data dan missing values
df.head()
```

```
[2]:
```

|   | panjang_tubuh | lebar_tubuh | panjang_sirip | lebar_sirip | spesies   |
|---|---------------|-------------|---------------|-------------|-----------|
| 0 | 5.1           | 3.5         | 1.4           | 0.2         | Ikan Nila |
| 1 | 4.9           | 3.0         | 1.4           | 0.2         | Ikan Nila |
| 2 | 4.7           | 3.2         | 1.3           | 0.2         | Ikan Nila |
| 3 | 4.6           | 3.1         | 1.5           | 0.2         | Ikan Nila |
| 4 | 5.0           | 3.6         | 1.4           | 0.2         | Ikan Nila |

2. Pisahkan antara atribut (fitur) dan label (spesies ikan).

```
[4]: # X: Mengambil semua kolom kecuali 'spesies' sebagai fitur
X = df.drop(columns=['spesies']).values

# Menyimpan nama-nama fitur dan nama spesies untuk visualisasi
feature_names = df.drop(columns=['spesies']).columns.tolist()
target_names = df['spesies'].unique().tolist()

# Mengubah kolom 'spesies' dari teks menjadi tipe kategori, lalu diubah menjadi angka
df['spesies'] = df['spesies'].astype('category')
y = df['spesies'].cat.codes

# Cek kembali data setelah spesies diubah menjadi angka
df.head()
```

Output:

```
[4]:
```

|   | panjang_tubuh | lebar_tubuh | panjang_sirip | lebar_sirip | spesies   |
|---|---------------|-------------|---------------|-------------|-----------|
| 0 | 5.1           | 3.5         | 1.4           | 0.2         | Ikan Nila |
| 1 | 4.9           | 3.0         | 1.4           | 0.2         | Ikan Nila |
| 2 | 4.7           | 3.2         | 1.3           | 0.2         | Ikan Nila |
| 3 | 4.6           | 3.1         | 1.5           | 0.2         | Ikan Nila |
| 4 | 5.0           | 3.6         | 1.4           | 0.2         | Ikan Nila |

Kemudian split data test dan train beserta output

#### ▼ Memisahkan data latih dan data uji + data scaling

```
[5]: # Membagi data: 80% untuk latih (training), 20% untuk uji (testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Inisialisasi fungsi scaling
scaler = StandardScaler()

# Melakukan scaling pada data latih dan uji
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Menampilkan jumlah data yang berhasil dibagi
print(f"Jumlah data training: {X_train.shape[0]}")
print(f"Jumlah data testing : {X_test.shape[0]}")

Jumlah data training: 120
Jumlah data testing : 30
```

### 3. Buat dan latih model Decision Tree.

```
[6]: # Inisialisasi dan latih model Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_scaled, y_train)

print("Model berhasil dilatih.")

Model berhasil dilatih.
```

```
[7]: # Lakukan prediksi pada data latih dan data uji
y_train_pred_dt = dt_model.predict(X_train_scaled)
y_test_pred_dt = dt_model.predict(X_test_scaled)

print("\nLaporan Klasifikasi Decision Tree (Data Testing):")
print(classification_report(y_test, y_test_pred_dt, target_names=target_names))
```

```
Laporan Klasifikasi Decision Tree (Data Testing):
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Ikan Nila    | 1.00      | 1.00   | 1.00     | 11      |
| Ikan Mas     | 1.00      | 1.00   | 1.00     | 9       |
| Ikan Lele    | 1.00      | 1.00   | 1.00     | 10      |
| accuracy     |           |        | 1.00     | 30      |
| macro avg    | 1.00      | 1.00   | 1.00     | 30      |
| weighted avg | 1.00      | 1.00   | 1.00     | 30      |

#### 4. Lakukan prediksi dengan model yang telah dilatih.

```
[8]: try:
panjang_tubuh = float(input("Masukkan Panjang Tubuh: "))
lebar_tubuh = float(input("Masukkan Lebar Tubuh: "))
panjang_sirip = float(input("Masukkan Panjang Sirip: "))
lebar_sirip = float(input("Masukkan Lebar Sirip: "))

# Gabungkan input menjadi array 2D
data_baru = np.array([[panjang_tubuh, lebar_tubuh, panjang_sirip, lebar_sirip]])

# Scale data baru menggunakan scaler yang sama dengan data training
data_baru_scaled = scaler.transform(data_baru)

# Prediksi menggunakan model Decision Tree
hasil_prediksi = dt_model.predict(data_baru_scaled)

# Ambil nama kategori (teks) berdasarkan hasil spesies
nama_spesies = target_names[hasil_prediksi[0]]

print(f"\n[HASIL PREDIKSI]: Ikan tersebut termasuk dalam spesies: {nama_spesies}")

except ValueError:
    print("Error: Mohon masukkan angka yang valid.")
```

Output:

```
Masukkan Panjang Tubuh: 5.3
Masukkan Lebar Tubuh: 3.2
Masukkan Panjang Sirip: 1.3
Masukkan Lebar Sirip: 0.3
```

```
[HASIL PREDIKSI]: Ikan tersebut termasuk dalam spesies: Ikan Lele
```

#### 5. Hitung akurasi model.

```
[9]: # Evaluasi akurasi Decision Tree
print("--- EVALUASI DECISION TREE ---")
print("Akurasi pada data training:", accuracy_score(y_train, y_train_pred_dt))
print("Akurasi pada data testing :", accuracy_score(y_test, y_test_pred_dt))

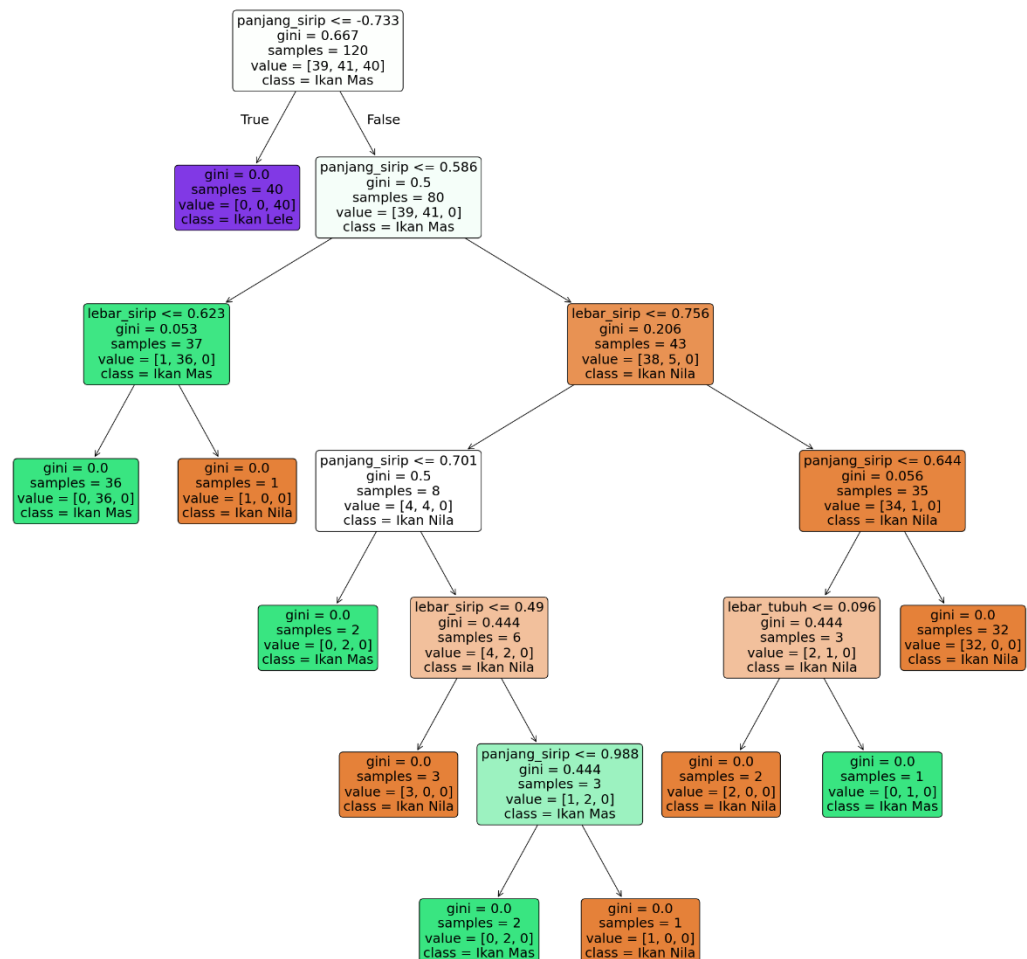
--- EVALUASI DECISION TREE ---
Akurasi pada data training: 1.0
Akurasi pada data testing : 1.0
```

6. Visualisasi struktur daun keputusan model Decision Tree yang telah dilatih.

```
[10]: plt.figure(figsize=(20, 20))
      plot_tree(dt_model,
                feature_names=feature_names,
                class_names=target_names,
                filled=True,
                rounded=True,
                fontsize=18)
      plt.title("Visualisasi Klasifikasi Spesies Ikan Menggunakan Decision Tree", fontsize=30)
      plt.tight_layout()
      plt.show()
```

Output:

Visualisasi Klasifikasi Spesies Ikan Menggunakan Decision Tree



7. Tarik kesimpulan yang diperoleh dari model.

Jawab:

Berdasarkan evaluasi model *Decision Tree*, diperoleh tingkat akurasi sempurna yakni 100% pada data pengujian. Meskipun secara metrik terlihat sangat baik, analisis lebih mendalam terhadap visualisasi struktur pohon menunjukkan adanya indikasi *overfitting* yang cukup kuat. Model terlihat

membentuk aturan percabangan yang terlalu spesifik dan cenderung ‘menghafal’ data latih secara presisi. Akibatnya, pemodelan ini berisiko kehilangan kemampuannya untuk melakukan generalisasi secara akurat ketika dihadapkan pada data pengukuran spesies ikan yang baru atau belum pernah dilihat sebelumnya di dunia nyata.

2. Dataset "ikan.csv" akan digunakan untuk membandingkan metode klasifikasi lain yang ada pada file referensi, yaitu SVM. Karena SVM sangat sensitif terhadap rentang nilai fitur, diperlukan tahap standarisasi data sebelum melatih model. Tahapan yang ada pada tugas ini antara lain:

1. Ubah dataset ke dalam dataframe.

```
[2]: # Membaca file CSV
df = pd.read_csv('ikan.csv')

# Menampilkan 5 baris pertama untuk melihat data dan missing values
df.head()
```

```
[2]:
```

|   | panjang_tubuh | lebar_tubuh | panjang_sirip | lebar_sirip | spesies   |
|---|---------------|-------------|---------------|-------------|-----------|
| 0 | 5.1           | 3.5         | 1.4           | 0.2         | Ikan Nila |
| 1 | 4.9           | 3.0         | 1.4           | 0.2         | Ikan Nila |
| 2 | 4.7           | 3.2         | 1.3           | 0.2         | Ikan Nila |
| 3 | 4.6           | 3.1         | 1.5           | 0.2         | Ikan Nila |
| 4 | 5.0           | 3.6         | 1.4           | 0.2         | Ikan Nila |

2. Pisahkan antara atribut dan label.

```
[4]: # X: Mengambil semua kolom kecuali 'spesies' sebagai fitur
X = df.drop(columns=['spesies']).values

# Menyimpan nama-nama fitur dan nama spesies untuk visualisasi
feature_names = df.drop(columns=['spesies']).columns.tolist()
target_names = df['spesies'].unique().tolist()

# Mengubah kolom 'spesies' dari teks menjadi tipe kategori, lalu diubah menjadi angka
df['spesies'] = df['spesies'].astype('category')
y = df['spesies'].cat.codes

# Cek kembali data setelah spesies diubah menjadi angka
df.head()
```

Output:

```
[4]:
```

|   | panjang_tubuh | lebar_tubuh | panjang_sirip | lebar_sirip | spesies   |
|---|---------------|-------------|---------------|-------------|-----------|
| 0 | 5.1           | 3.5         | 1.4           | 0.2         | Ikan Nila |
| 1 | 4.9           | 3.0         | 1.4           | 0.2         | Ikan Nila |
| 2 | 4.7           | 3.2         | 1.3           | 0.2         | Ikan Nila |
| 3 | 4.6           | 3.1         | 1.5           | 0.2         | Ikan Nila |
| 4 | 5.0           | 3.6         | 1.4           | 0.2         | Ikan Nila |

3. Lakukan Feature Scaling menggunakan StandardScaler agar semua fitur memiliki rentang nilai yang seragam.

▼ Memisahkan data latih dan data uji + data scaling

```
[5]: # Membagi data: 80% untuk latih (training), 20% untuk uji (testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Inisialisasi fungsi scaling
scaler = StandardScaler()

# Melakukan scaling pada data latih dan uji
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Menampilkan jumlah data yang berhasil dibagi
print(f"Jumlah data training: {X_train.shape[0]}")
print(f"Jumlah data testing : {X_test.shape[0]}")

Jumlah data training: 120
Jumlah data testing : 30
```

4. Buat dan latih model SVM (Support Vector Classifier) dengan menggunakan kernel linear.

```
[12]: # Inisialisasi dan latih model SVM dengan kernel Linear
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train_scaled, y_train)
```

```
[12]:
```

▼ SVC ⓘ ?

► Parameters

5. Lakukan prediksi pada data pengujian.

```
[13]: # Lakukan prediksi pada data latih dan data uji
y_train_pred_svm = svm_model.predict(X_train_scaled)
y_test_pred_svm = svm_model.predict(X_test_scaled)

# Evaluasi akurasi SVM
print("--- EVALUASI SVM ---")
print("Akurasi pada data training:", accuracy_score(y_train, y_train_pred_svm))
print("Akurasi pada data testing :", accuracy_score(y_test, y_test_pred_svm))

print("\nLaporan Klasifikasi pada Data Testing:")
print(classification_report(y_test, y_test_pred_svm, target_names=target_names))
```

Output:

```
--- EVALUASI SVM ---
Akurasi pada data training: 0.9833333333333333
Akurasi pada data testing : 0.9666666666666667

Laporan Klasifikasi pada Data Testing:
              precision    recall  f1-score   support

   Ikan Nila         0.92         1.00         0.96         11
    Ikan Mas         1.00         0.89         0.94          9
    Ikan Lele         1.00         1.00         1.00         10

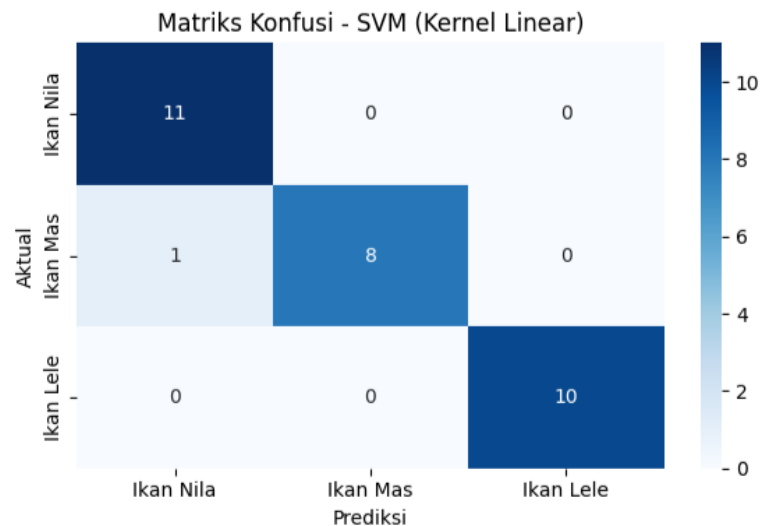
 accuracy                   0.97         30
 macro avg              0.97         0.96         0.97         30
 weighted avg           0.97         0.97         0.97         30
```

- Evaluasi model dengan menampilkan Confusion Matrix menggunakan (Seaborn) untuk melihat spesies jumlah prediksi yang benar dalam satu dataset.

```
[14]: # Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred_svm)

# Membuat visualisasi heatmap dari confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=target_names,
            yticklabels=target_names)
plt.xlabel('Prediksi')
plt.ylabel('Aktual')
plt.title('Matriks Konfusi - SVM (Kernel Linear)')
plt.tight_layout()
plt.show()
```

Output:



7. Tarik kesimpulan mengenai performa model SVM dalam mengklasifikasikan spesies ikan berdasarkan hasil evaluasi.

Jawab:

Berdasarkan pengujian pemodelan *Support Vector Machine* (SVM), model menghasilkan tingkat akurasi sebesar 97%. Walaupun secara persentase sedikit di bawah *Decision Tree*, hasil observasi melalui *Confusion Matrix* membuktikan bahwa SVM bekerja jauh lebih tangguh dan logis. Model ini hanya mencatatkan satu insiden misklasifikasi yang masih dalam batas kewajaran, yakni saat memprediksi Ikan Mas sebagai Ikan Nila. Hal ini menunjukkan bahwa algoritma SVM mampu membangun *decision boundary* yang lebih natural, tidak terjebak pada pola *overfitting*, dan jauh lebih dapat diandalkan untuk skenario klasifikasi spesies baru di masa mendatang.